Now you know how to convert a block to a Proc and back again. Let's look at a real-life example of a callable object that starts its life as a lambda and is then converted to a regular block.

The HighLine Example

The HighLine gem helps you automate console input and output. For example, you can tell HighLine to collect comma-separated user input and split it into an array, all in a single call. Here's a Ruby program that lets you input a comma-separated list of friends:

blocks/highline_example.rb

```
require 'highline'

hl = HighLine.new
friends = hl.ask("Friends?", lambda {|s| s.split(',') })
puts "You're friends with: #{friends.inspect}"
```

```
Friends?
Ivana, Roberto, Olaf
You're friends with: ["Ivana", " Roberto", " Olaf"]
```

You call HighLine#ask with a string (the question for the user) and a Proc that contains the post-processing code. (You might wonder why HighLine requires a Proc argument rather than a simple block. Actually, you *can* pass a block to ask, but that mechanism is reserved for a different HighLine feature.)

If you read the code of HighLine#ask, you'll see that it passes the Proc to an object of class Question, which stores the Proc as an instance variable. Later, after collecting the user's input, the Question passes the input to the stored Proc.

If you want to do something else to the user's input—say, change it to uppercase—you just create a different Proc:

```
name = hl.ask("Name?", lambda {|s| s.capitalize })
puts "Hello, #{name}"
```

```
Name?
bill
Hello, Bill
```

This is an example of *Deferred Evaluation* (89).

Procs vs. Lambdas

You've learned a bunch of different ways to turn a block into a Proc: Proc.new, lambda, the & operator.... In all cases, the resulting object is a Proc.